Curtin University — Discipline of Computing

Unix & C Programming (COMP1000)

Semester 1, 2026

Assignment

Due: Wednesday, 20 May 2026, 11:59 PM (GMT+8)

Weight: 30% of the unit

Your task for this assignment is to design, code (in C89 version of C) and test a program. In summary, your program will:

- Able to create dynamically-allocated 2D char array to make a simple ASCII-based game.
- Receive user input to control the game.
- Use pre-written colour generator to add background/foreground colour in the game.
- Use pre-written random number generator to control the Enemy movement.
- Write a proper makefile. Without the makefile, we will assume it cannot be compiled and it will negatively affect your mark.
- Extract information from a text file to initialize the game.
- Utilise linkedlist data structure to keep track the game progress, allowing the Player to undo the steps taken.

1 Code Design

You must comment your code sufficiently in a way that we understand the overall design of your program. Remember that you cannot use "//" to comment since it is C99-specific syntax. You can only use "/*.....*/" syntax.

Your code should be structured in a way that each file has a clear scope and goal. For example, "main.c" should only contain a main function, "game.c" should contain functions that handle and control the game features, and so on. Basically, functions should be located on reasonably appropriate files and you will link them when you write the makefile. DO NOT put everything together into one single source code file. Make sure you use the header files and header guard correctly. NEVER #include .c files directly, instead #include .h files only.

Make sure you free all the memories allocated by the malloc() function. Use valgrind to detect any memory leak and fix them. Memory leaks might not break your program, but penalty will still be applied if there is any. If you do not use malloc at all, you will lose marks.

Please be aware of our coding standard (can be found on Blackboard and the marking criterias) Violation of the coding standard will result in penalty on the assignment. Keep in mind that it is possible to lose significant marks with a fully-working program that is written messily,

has no clear structure, full of memory leaks, and violates many of the coding standards. The purpose of this unit is to teach you a good habit of programming.

2 Academic Integrity

This is an assessable task, and as such there are strict rules. You must not ask for or accept help from anyone else on completing the tasks. You must not show your work at any time to another student enrolled in this unit who might gain unfair advantage from it. These things are considered plagiarism or collusion. To be on the safe side, never release your code to the public.

Do NOT just copy some C source codes from internet resources (including AI tools such as ChatGPT). If you get caught, it will be considered plagiarism. If you put the reference, then that part of the code will not be marked since it is not your work.

Staff can provide assistance in helping you understand the unit material in general, but nobody is allowed to help you solve the specific problems posed in this assignment. The purpose of the assignment is for you to solve them on your own, so that you learn from it. Please see Curtin's Academic Integrity website for information on academic misconduct (which includes plagiarism and collusion).

The unit coordinator may require you to attend quick interview to answer questions about any piece of written work submitted in this unit. Your response(s) may be referred to as evidence in an Academic Misconduct inquiry. In addition, your assignment submission may be analysed by systems to detect plagiarism and/or collusion.

3 Task Details

3.1 Quick Preview

First of all, please watch the supplementary videos on the Assignment link on Blackboard. These videos demonstrates what we expect your program should do. You will implement a simple game that you can play on Linux terminal. Further details on the specification will be explained on the oncoming sections.

3.2 Command Line Arguments

Please ensure that the executable is called **"labyrinth"**. Your executable should accept only one more command-line parameter/argument:

./labyrinth <map_file>

<map_file> is the text file name containing the information to initiate your game. It is your responsibility to check whether the file can be opened properly. You can assume the content of the file is valid to initiate the game.

If the amount of the arguments are too many or too few, your program should print a message on the terminal and end the program accordingly (do NOT use exit() function).

Note: Remember, the name of the executable is stored in variable argv[0].

3.3 Main Interface

Once you run the program, it should clear the terminal screen (watch the video about the method to clear the screen) and print the 2D map containing all the objects: Walls, Goal, Treasure, Player, and Enemy:

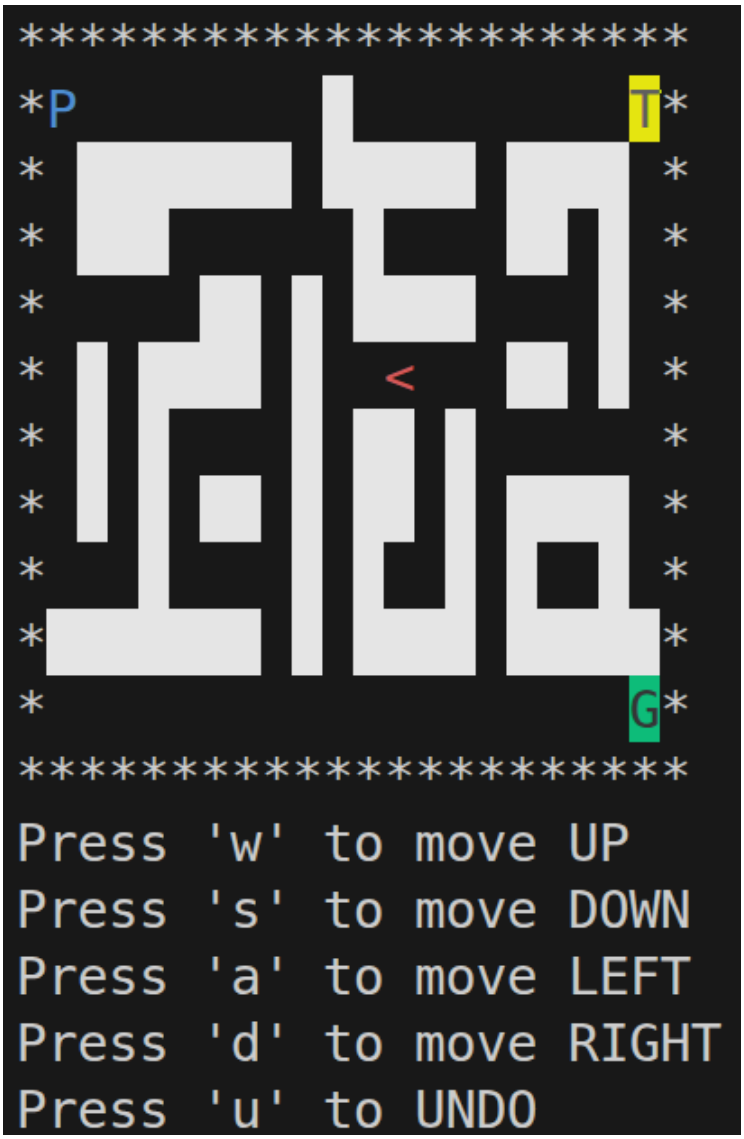


Figure 1: Main visual interface of the game.

You can type the command to move the Player. Player and Enemy are not able to go through the Walls and the map borders. Every time the user inputs the command, the program should update the map accordingly and refresh the screen (clear the screen and reprint the map). Please refer to Section 3.6.1 for more detail on the user input.

3.4 Characters on the Map

You have to use the correct characters on the game interface:

- Char '*****' for the map border.
- An empty space ' ' char for the Wall.
- Char '**G**' for the Goal.
- Char '**T**' for the Treasure.
- Char '**P**' for the Player.
- Char '**^**' or '**>**' or '**v**' or '**<**' for the Enemy.

Furthermore, some objects have **foreground colour**:

- **Player** has **blue** foreground colour.
- **Enemy** has **red** foreground colour. (Before Player takes the Treasure)

Some objects have **background colour**:

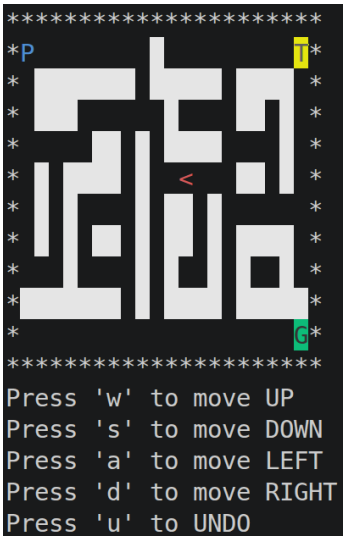
- **Wall** has **white** background colour.
- **Goal** has **green** background colour.
- **Treasure** has **yellow** background colour.
- **Enemy** has **red** background colour. (After Player takes the Treasure. Font color returns to white when Enemy is furious.)

3.5 File I/O

Please watch the supplementary video about the File I/O. The text file will look like this:

```
10 20
4 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 3
0 1 1 1 1 1 1 1 0 1 1 1 1 1 0 1 1 1 1 0
0 1 1 1 0 0 0 0 0 0 1 0 0 0 0 1 1 0 1 0
0 0 0 0 0 1 1 0 1 0 1 1 1 1 0 0 0 0 1 0
0 1 0 1 1 1 1 0 1 0 0 5 0 0 0 1 1 0 1 0
0 1 0 1 0 0 0 0 1 0 1 1 0 1 0 0 0 0 0 0
0 1 0 1 0 1 1 0 1 0 1 1 0 1 0 1 1 1 1 0
0 0 0 1 0 0 0 0 1 0 1 0 0 1 0 1 0 0 1 0
1 1 1 1 1 1 1 0 1 0 1 1 1 1 0 1 1 1 1 1
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 2
```

(a) content of map text file



(b) corresponding interface

Figure 2: This is an example of the map text file and the game interface created from it.

The two integers on the first line are the playable **row** and **column** map size respectively. These map sizes do not include the border surrounding the map. Afterwards, the remaining lines are the content of the 2D map array. You can assume the text file contains valid information with valid datatype. Each integer represents:

- 0 represents ' ' (An empty space)
- 1 represents ' ' (Walls, also an empty space, but with white background)
- 2 represents 'G' (Goal)
- 3 represents 'T' (Treasure)
- 4 represents 'P' (Player)
- 5 represents '<' (Enemy always facing left at the start of the game)

Note: Keep in mind that we will use our own map text file when we mark your assignment. So, please make sure your file I/O implementation works with various map files. Feel free to create your own map file.

3.6 Gameplay

Please watch the supplementary videos for visual demonstration. The main objective is to move the Player to grab the Treasure, and then escape through the Goal. There is an Enemy that is patrolling around the area. The Enemy moves twice for every Player movement. When the Treasure is taken by the Player, the Enemy will be furious and move three times on every turn.

3.6.1 User Input

The user only needs to type 1 character for each command (lower case). Here is the list of the possible commands:

- **'w'** moves the Player one block above.
- **'s'** moves the Player one block below.
- **'a'** moves the Player one block left.
- **'d'** moves the Player one block right.
- **'u'** undoes the movement of the Player and the Enemy. This also undoes the Treasure looting.
- Any other character should be ignored, and nothing changes on the map. Feel free to print a warning message such as "invalid key".

Your program should be able to receive each input without pressing "enter" key everytime. Please refer to the supplementary video for a sample code to disable the Echo and Canonical feature on Linux terminal temporarily. If your terminal is stuck on this mode, you only need to re-open a new terminal.

Note: Every action should update the 2D map array, clear the terminal screen and reprint the map.

3.6.2 Winning/Losing the Game

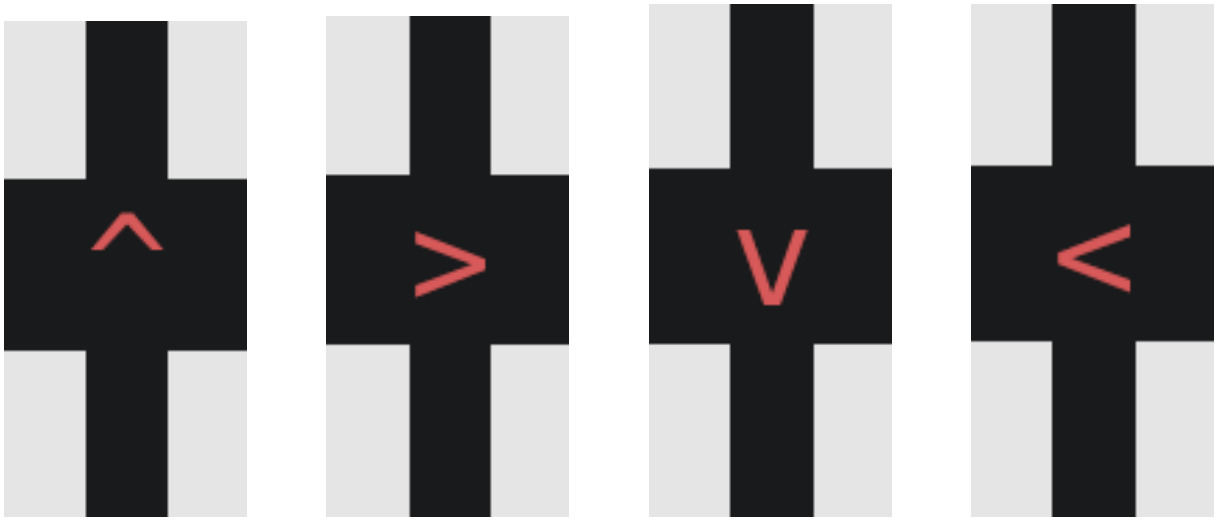
The game will continue playing until either the Player manages to **loot the Treasure, and then reach the Goal** (**winning**) OR the Player **is caught by the Enemy**. You can print message such as "You win!" or "You lose!" when the game ended. Please do **NOT** use **exit**() or **break** to end the program. You must free all the malloc'ed memories and ensure the program reaches the end of the main function properly.

Note: If the Player has not yet collected the Treasure, the Goal behaves like a Wall. The Player cannot reach the Goal without first obtaining the Treasure.

3.6.3 Enemy Movement Algorithm

The Enemy moves in a specific algorithm. Keep in mind that the Enemy is NOT actively seeking the direct path to the Player (doing so will make the game impossible to win since the Enemy moves faster). Here are the details:

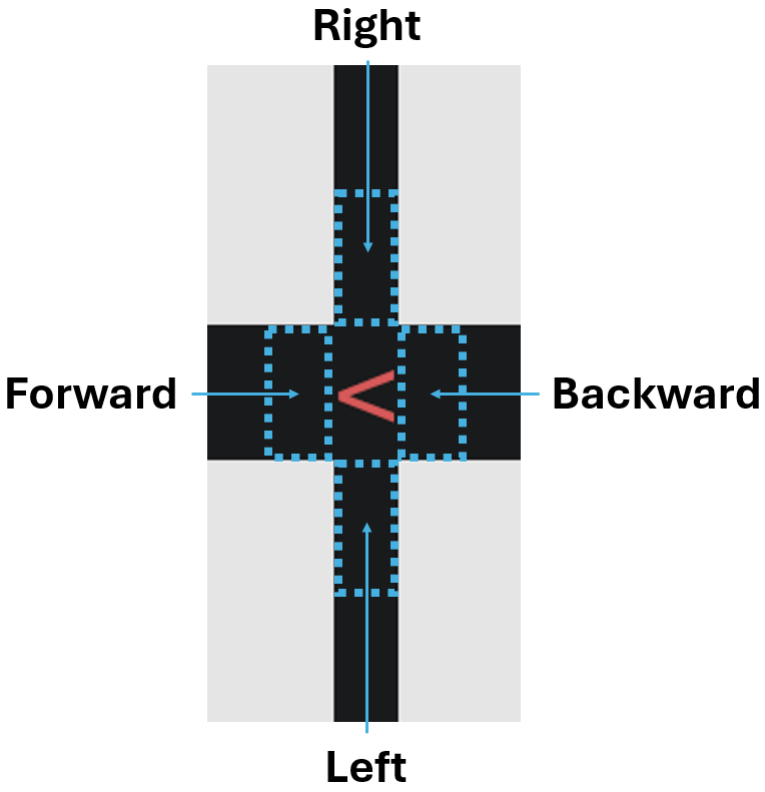
- The Enemy can face four directions
  - '^' for facing UP.
  - '>' for facing RIGHT
  - 'v' for facing DOWN
  - '<' for facing LEFT



(a) UP (b) RIGHT (c) DOWN (d) LEFT

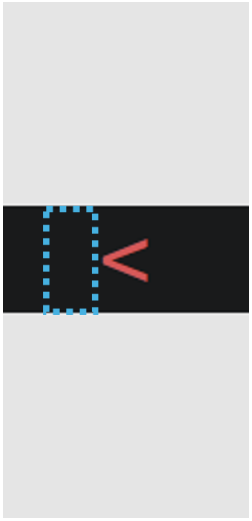
Your program need to change the character depending on the Enemy movement.

- The term **"forward"** refers to the direction the enemy is facing. For example, if the enemy is currently facing left, **moving forward** means the Enemy is moving one block to the direction it is facing. On the other hand, the term **"backward"** is the opposite direction. Lastly, the terms **"left"** and **"right"** represent the block on the Enemy's left and right sides respectively.



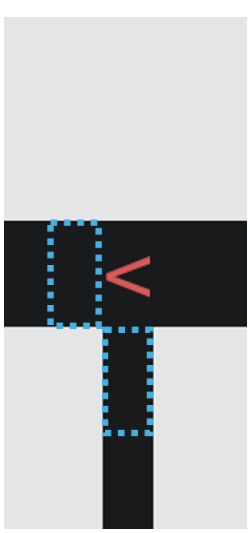
These are the relative direction terminologies based on where the Enemy is facing currently.

- As mentioned previously, the Enemy can move twice on every Player's turn (or three times in a row in a furious state when the Treasure has been looted by the Player). Keep in mind that the Enemy treats the Treasure and Goal as a Wall. These are the details on how it moves on every step:
  - If the way forward is free, BUT both left and right paths are blocked, then it can only move forward.

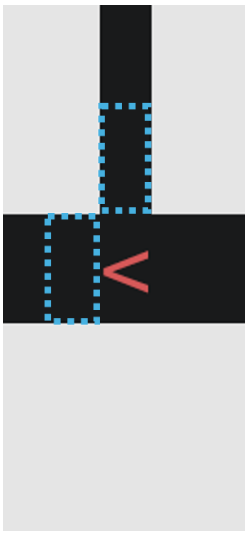


The dashed blue box represents the possible movement.

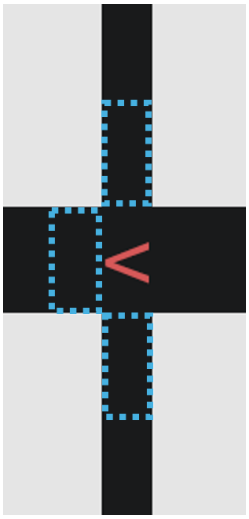
  - If the way forward is free, AND at least one of the side paths (left and right) is free, this is considered as an intersection. In this case, use **Random Number Generator (RNG)** to choose which path to choose. Choosing the left path will rotate the Enemy by $90°$ **anti-clockwise**. On the other hand, choosing the right path will rotate the Enemy by $90°$ **clockwise**. Please change the Enemy character accordingly.



(a)

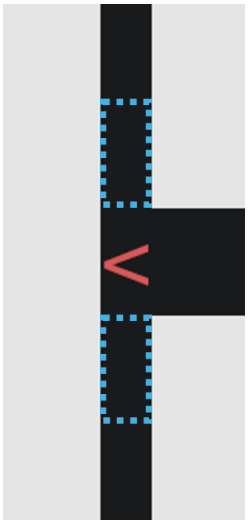


(b)

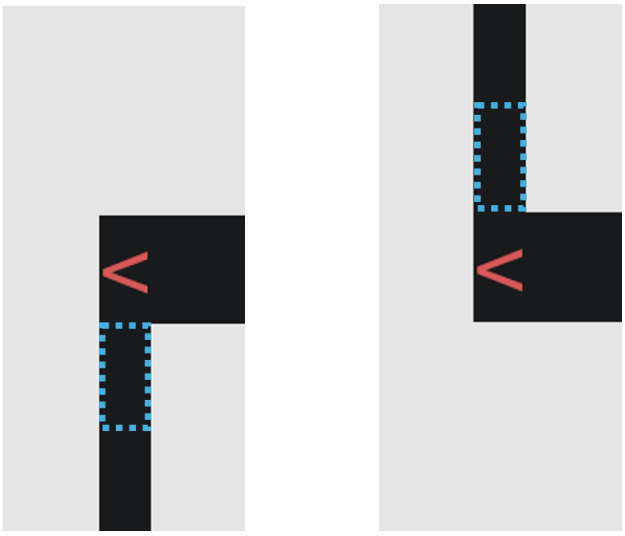


(c)

– If the way forward is blocked (*e.g* by Wall, map border, Treasure, Goal), AND both of the side paths (left and right) are free, use **RNG** to choose the next path.

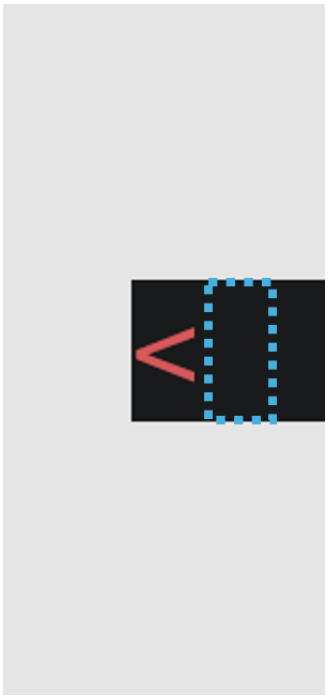


– If the way forward is blocked, AND only ONE side path is free (left or right), then it moves to that path and rotate accordingly.



(a) (b)

– If the way forward, left, and right paths are ALL blocked, then it rotates $180°$ and moves to the opposite direction.



Note: To summarise the algorithm of the Enemy movement, it will keep moving forward along the corridor until it meets an intersection. On any intersection, use the **Random Number Generator (RNG)** to decide which path to choose. If the Enemy choose the right side, then its orientation will rotate $90°$ clockwise. If it chooses the left side, it will rotate $90°$ anti-clockwise. Lastly, if it meets a complete dead end (no way forward, left, and right), then it will rotate $180°$ and move to the other direction.

3.7 Struct and Generic Linked List to implement UNDO feature

In this section, you will need to write a **generic linked list** with appropriate struct to implement **UNDO** feature. This feature is triggered when the user enters the key 'u'. Everytime the key is pressed, the Player and Enemy will move back to their previous position. It is also possible to undo the Treasure looting and make it appear again. The Player can use the UNDO feature at any point of the game as long as the game is not over (Not losing AND not winning yet). It should be possible to keep UNDO-ing up to the initial state of the game.

Please refer to supplementary video for some advices regarding this implementation. The summary is that you can use the linkedlist node to store any information that you deem necessary. For example, you can store the previous Player 'P' location (This is only one example, you can store more information if you need it). You will need to create a new linkedlist node every time the Player attempts to move (with malloc()) and store it in the linkedlist. When the Player attempts to do invalid move (*e.g* moving to a Wall OR outside map border), it does not count as a movement, and the linkedlist should not record that move.

When the Player presses 'u', your program should be able to retrieve the most recent node and update the map to the previous state. If game state is back to the initial configuration (when you just started the game), pressing 'u' should **NOT** do anything. One possible way to confirm this is by checking if the linkedlist is empty. (This depends on your implementation. Implement what you need to achieve this task.)

Note: To get full mark on this section, you need to implement a fully generic linkedlist. If you implement a non-generic linkedlist, you still can get partial mark. Please do not just copy the whole linkedlist code from someone else because it might lead to collusion academic misconduct.

3.8 Makefile

You should manually write the makefile according to the format explained in the lecture and practical. It should include the Make variables, appropriate flags, appropriate targets, correct prerequisites, and clean rule. We will compile your program through the makefile. If the makefile is missing, we will assume the program cannot be compiled, and you will lose marks. **DO NOT** use the makefile that is auto-generated by the VScode. Write it yourself.

3.9 Assumptions

For simplification purpose, these are assumptions you can use for this assignments:

- The content of the map files are valid. (If there is NO error when opening it)
- The size of the map in the text file is at least 10 rows AND at least 10 columns.
- All game objects exist. There will be ONLY one Player, ONLY one Goal, ONLY one Treasure, and ONLY one Enemy. There will be at least one Walls.
- All corridors formed by the walls are one block wide.
- There will be a winning path from the Player to the Treasure and Goal. (Not in isolation)
- The Enemy is not enclosed by the walls. There is a path for the Enemy to reach the Player.
- The size of the map will be reasonable to be displayed on terminal. For example, we will not test the map with gigantic size such as 300 x 500.
- You only need to handle lowercase inputs ('w', 's', 'a', 'd', 'u'). The other keys should be ignored (feel free to add warning message).

Note: When you create your own custom maps for testing, please make sure to follow these assumptions.

4 Marking Criteria

This is the marking distribution for your guidance:

- Properly structured makefile according to the lecture and practical content (**5 marks**)
- Program can be compiled with the makefile and executed successfully without immediate crashing and showing reasonable output (**5 marks**)
- Usage of header guards and reasonable in-code commenting (**2 marks**)
- The whole program is readable and has reasonable framework. Multiple files are utilized with reasonable category. If you only have one c file for the whole assignment (not counting terminal.c, newsleep.c, and random.c), you will get zero mark on this category. (**3 marks**)
- Avoiding bad coding standard. (**5 marks**) Please refer to the coding standard on Blackboard. Some of the most common mistakes are:
  - Using global variables
  - Calling exit() function to end the program abruptly
  - Using "break'' NOT on the switch case statement
  - Using "continue''
  - Using goto
  - Having multiple returns on a single function
  - #include the .c files directly instead of the .h files
- No memory leaks (**10 marks**) Please use valgrind command to check for any memory leak. If your program is very sparse OR does not use any malloc(), you will get zero mark on this category.

- **FUNCTIONALITIES**:
  - Correct command line arguments verification with proper response. This includes detecting the success/failure of opening the text file. **(5 marks)**
  - File IO is done successfully to retrieve all game information from the map text file. Proper memory allocation for the 2D map array. **(10 marks)**
  - Able to clear the screen and re-print the map (with correct symbols and correct foreground and background colours for all objects) on every action. **(10 marks)**
  - Able to move the Player with the keyboard input from the user without pressing 'Enter' key. (and cannot go through Wall and map border) **(5 marks)**
  - Winning when the Player reaches the Goal (after obtaining Treasure) AND Losing when the Enemy reaches the Player. **(5 marks)**
  - The Enemy moves according to the algorithm described in section 3.6.3. This includes the multiple Enemy movements for every Player's turn (and furious state as well) and orientation change. **(20 marks)**
  - Able to UNDO the game utilizing linkedlist. **(15 marks)** (If linkedlist is NOT generic, only 10 marks max)

Note: Remember, if your program fails to demonstrate that the functionality works, then the mark for that functionality will be capped at 50%. If your program does not compile at all OR just crashed immediately upon running it, then the mark for all the functionalities will be capped at 50% (For this assignment, it means maximum of 35 out of 70 marks on functionalities). You then need describe your code clearly on the short report on the next section.

5 Short Report

If your program is incomplete/imperfect, please write a short report explaining what you have done on for each functionality. Inform us which function on which file that is related to that functionality. This report is NOT marked, but it will help us a lot to mark your assignment. Poorly-written report will not help us to understand your code. Please ensure your report reflects your work correctly (no exaggeration). Dishonest report will lead to academic misconduct.

6 Final Check and Submission

After you complete your assignment, please make sure it can be compiled and run on our Linux lab environment. If you do your assignment on other environments (e.g on Windows operating system), then it is your responsibility to ensure that it works on the lab environment. In the case of incompatibility, it will be considered "not working'' and some penalties will apply. You have to submit a **single zip file** containing ALL the files in this list in **ONE FOLDER**:

- **Declaration of Originality Form** – Please fill this form digitally and submit it. This is an important document for assignment submission.
- **Your essential assignment files** – Submit all the .c & .h files and your makefile. Please do not submit the executable and object (.o) files, as we will re-compile them anyway. Do not create any sub-directory within the directory. Every file should exist within the same directory.
- **Brief Report** (if applicable) - If you want to write the report about what you have done on the assignment, please save it as a PDF or TXT file.

Note: Please compress all the necessary files into a **SINGLE zip file**. So, your submission should only has one zip file. If you do not compress the files, we reserve the right to refuse your submission. If you get an **extension**, please **do NOT submit the early draft** to the Blackboard because we might accidentally mark the earlier version of your work. Only submit to Blackboard once you have the final version of your assignment.

The name of the zipped file should be in the format of:

<your-tutor-name>_<student-ID>_<your-full-name>_Assignment.zip
Example: Antoni_12345678_Michael-Bay_Assignment.zip

If you forget your tutor's name, please put the lecturer's name. Please make sure your submission is complete and not corrupted. You can re-download the submission and check if you can compile and run the program again. Corrupted submission will receive instant zero. Late submission will receive zero mark. You can submit it multiple times, but only your latest submission will be marked. Therefore, please make sure the latest submission is the complete code.

End of Assignment